

INDEX.HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cheapong</title>

  <link rel="stylesheet" href="./css/style.css">
</head>
<body>

  <main class="cheapong-page">
    <section class="cheapong-wrapper">
      <h1>Cheapong</h1>

      <canvas id="gameCanvas">
        Tu navegador no soporta canvas.
      </canvas>
    </section>
  </main>

  <script type="module" src="./js/main.js"></script>
</body>
</html>
```

CSS/style.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background-color: #111;
  font-family: Arial, sans-serif;
  color: white;
  overflow: hidden;
}

.cheapong-page {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.cheapong-wrapper {
```

```
    text-align: center;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 1rem;
}

.cheapong-wrapper h1 {
    margin: 0;
    font-size: clamp(2rem, 6vw, 3rem);
}

#gameCanvas {
    width: min(90vw, calc(75vh * 1.3333), 50rem);
    aspect-ratio: 4 / 3;
    height: auto;
    background-color: black;
    border: 0.125rem solid white;
    display: block;
}
```

JS/main.js

```
/**
 * Cheapong - Main entry point
 *
 * - get canvas
 * - get 2D context
 * - draw initial screen
 */

const canvas = document.getElementById("gameCanvas");

if (!canvas) { throw new Error("Canvas element not found"); }

const ctx = canvas.getContext("2d");

const canvasWidth = canvas.width;
const canvasHeight = canvas.height;

// Basic application states

const AppState = Object.freeze({
    MAIN_MENU: "MAIN_MENU",
    IN_GAME: "IN_GAME"
});

// Current applicaton state
let appState = AppState.MAIN_MENU;

const score = {
```

```
    player: 0,
    cpu: 0
  }

  const GameState = Object.freeze({
    SERVE_PLAYER: "SERVE_PLAYER",
    SERVE_CPU: "SERVE_CPU",
    RALLY: "RALLY"
  });

  let gameState = GameState.SERVE_PLAYER;

  const inputState = {
    left: false,
    right: false
  }

  // Player paddle
  const playerPaddle = {
    x: (canvasWidth - 120) / 2,
    y: canvasHeight - 15 - 20,
    width: 120,
    height: 15,
    speed: 5
  };

  // CPU paddle
  const cpuPaddle = {
    x: (canvasWidth - 120) / 2,
    y: 20,
    width: 120,
    height: 15,
    speed: 5
  };

  function drawPaddle(paddle) {
    ctx.fillStyle = "white";

    ctx.fillRect(
      paddle.x,
      paddle.y,
      paddle.width,
      paddle.height
    );
  }

  // Ball
  const ball = {
    x: (canvasWidth - 20) / 2,
    y: (canvasHeight - 20) / 2,
    size: 20,
    speedX: 3,
    speedY: 3
  }
```

```
};

function drawBall() {
  const radius = ball.size / 2;
  const centerX = ball.x + radius;
  const centerY = ball.y + radius;

  ctx.fillStyle = "white";
  ctx.beginPath();
  ctx.arc(centerX, centerY, radius, 0, Math.PI * 2);
  ctx.fill();
}

function clearCanvas() {
  ctx.fillStyle = "black";
  ctx.fillRect(0, 0, canvasWidth, canvasHeight);
}

/**
 * Draws centered text on the canvas.
 *
 * @param {string} text - Text to draw.
 * @param {number} y - Vertical position.
 * @param {number} fontSize - Font size in pixels.
 */
function drawCenteredText(text, y, fontSize = 32) {
  ctx.fillStyle = "white";
  ctx.font = `${fontSize}px Arial`;
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";

  ctx.fillText(text, canvasWidth / 2, y);
}

/**
 * Renders the initial main menu.
 */
function renderMainMenu() {
  clearCanvas();
  drawCenteredText("CHEAPONG", canvasHeight * 0.30, 30);
  drawCenteredText("PRESS SPACE TO START", canvasHeight * 0.50, 16);
}

window.addEventListener("keydown", (event) => {
  if (event.code === "Space" && appState === AppState.MAIN_MENU) { appState =
AppState.IN_GAME; render(); }
  if (event.code === "Space" && appState === AppState.IN_GAME && gameState ===
GameState.SERVE_PLAYER) { launchPlayerServe(); }

  if (event.code === "ArrowLeft") { inputState.left = true; }
  if (event.code === "ArrowRight") { inputState.right = true; }
});

window.addEventListener("keyup", (event) => {
```

```
    if (event.code === "ArrowLeft") { inputState.left = false; }
    if (event.code === "ArrowRight") { inputState.right = false; }
  })

// Game Loop
function gameLoop() {
  render();
  requestAnimationFrame(gameLoop);
}

gameLoop();

/**
 * Renders a temporary game screen.
 */
function renderGameScreen() {
  clearCanvas();

  updatePlayerPaddle();

  if (gameState === GameState.SERVE_PLAYER) { updateServe(playerPaddle); }
  if (gameState === GameState.SERVE_CPU) { updateServe (cpuPaddle); }

  if (gameState === GameState.RALLY) {
    updateBall();
    handleWallCollision();
    handlePlayerPaddleCollision();
    handleCpuPaddleCollision();
  }

  drawPaddle(playerPaddle);
  drawPaddle(cpuPaddle);
  drawBall();

  drawCenteredText("GAME STARTED", canvasHeight * 0.40, 24);
  drawCenteredText("Cheapong game area", canvasHeight * 0.52, 16);
}

/**
 * Renders the current screen according to the app state.
 */
function render() {
  if (appState === AppState.MAIN_MENU) { renderMainMenu(); }
  if (appState === AppState.IN_GAME) { renderGameScreen(); }
}

function updatePlayerPaddle() {
  if (inputState.left) { playerPaddle.x -= playerPaddle.speed; }
  if (inputState.right) { playerPaddle.x += playerPaddle.speed; }

  // Borders
  if (playerPaddle.x < 0) { playerPaddle.x = 0; }
  if (playerPaddle.x + playerPaddle.width > canvasWidth) { playerPaddle.x =
canvasWidth - playerPaddle.width; }
```

```
}

function updateBall() {
  ball.x += ball.speedX;
  ball.y += ball.speedY;
}

function handleWallCollision() {
  if (ball.x <= 0) { ball.x = 0; ball.speedX *= -1; }
  if (ball.x + ball.size >= canvasWidth) { ball.x = canvasWidth - ball.size;
ball.speedX *= -1; }

  if (ball.y <= 0) { ball.y = 0; ball.speedY *= -1; }
  if (ball.y + ball.size >= canvasHeight) { ball.y = canvasHeight - ball.size;
ball.speedY *= -1; }
}

function handlePlayerPaddleCollision() {
  const isColliding = intersects(ball, playerPaddle);
  if (isColliding && ball.speedY > 0) { ball.y = playerPaddle.y - ball.size;
ball.speedY *= -1; }
}

function handleCpuPaddleCollision() {
  const isColliding = intersects(ball, cpuPaddle);
  if (isColliding && ball.speedY < 0) { ball.y = cpuPaddle.y + cpuPaddle.height;
ball.speedY *= -1; }
}

function intersects(ball, paddle) {
  return (
    ball.x < paddle.x + paddle.width &&
    ball.x + ball.size > paddle.x &&
    ball.y < paddle.y + paddle.height &&
    ball.y + ball.size > paddle.y
  );
}

function handleScore() {
  if (ball.y <= 0) { score.player++; resetBall(); }
  if (ball.y >= canvas.height) { score.cpu++; resetBall(); }
}

function updateServe(paddle) {
  ball.x = paddle.x + (paddle.width - ball.size) / 2;
  if (gameState === GameState.SERVE_PLAYER) { ball.y = playerPaddle.y -
ball.size; }
  if (gameState === GameState.SERVE_CPU) { ball.y = cpuPaddle.y +
cpuPaddle.height; }
}

function launchPlayerServe() {
  ball.speedX = 0;
}
```

```
    ball.speedY = 4.5;  
    gameState = GameState.RALLY;  
}
```